Statis: High-Performance Seed-Deterministic Userspace Supervision for Unmodified Linux Binaries

Siddid Pandey
Independent Systems Researcher
Kathmandu, Nepal
qvostwon@gmail.com

ABSTRACT

Non-determinism in concurrent software testing remains a fundamental challenge in systems engineering, leading to irreducible flake rates in distributed protocols, databases, and actor systems. Existing record-replay debuggers and deterministic execution frameworks either impose crippling performance overheads, require kernel modifications, or bloat execution traces to gigabytes per second. We present **Statis** (https://github.com/qvost/statis), a novel zero-overhead, seed-deterministic userspace runtime supervisor for unmodified Linux x86-64 multi-threaded ELF binaries. Statis neutralizes the five primary vectors of host non-determinism—wall-clock leakage, hardware timestamp counters, kernel entropy, address space layout randomization, and process identity virtualisation—at the binary boundary without source recompilation or kernel patches. Leveraging Seccomp BPF filters with new listener IPC, a monotonic priority time-wheel, and hardware PMU preemption, Statis achieves instant time-warping exceeding 500× speedups on idle-heavy workloads while maintaining a compact deterministic trace footprint measured in kilobytes rather than gigabytes.

KEYWORDS

Deterministic Execution, Runtime Supervision, Seccomp BPF, Time Warping, Fault Injection, Linux Systems

1 INTRODUCTION

Modern distributed systems, consensus engines (e.g., Raft, Paxos), and multi-threaded asynchronous runtimes rely heavily on deterministic testing to uncover subtle concurrency races and deadlocks. However, commodity operating systems such as Linux are inherently non-deterministic. Wall-clock reads, hardware timestamp counters (`rdtsc`), kernel entropy sources, asynchronous signal delivery, and preemptive thread scheduling introduce entropy that renders concurrency bugs non-reproducible.

Traditional approaches to deterministic execution occupy uncomfortable trade-off spaces. Record-and-replay frameworks like Mozilla `rr` [1] capture complete hardware execution traces but generate gigabytes of log data per second, rendering long-running distributed test suites storage-prohibitive. Virtual machine-based deterministic monitors (e.g., Determinator, Time-Traveling VMs) impose severe CPU overheads through full-system emulation or shadow page table maintenance. Kernel-level deterministic runtimes (e.g., Meta's `Hermit` or `DetTrace`) require custom kernel modules or container patches, breaking compatibility with standard cloud-native environments and unmodified enterprise ELF binaries.

We introduce **Statis**, a high-performance userspace runtime supervisor designed to execute unmodified Linux x86-64 multi-threaded ELF binaries under strict seed-determinism with zero performance overhead on hot execution paths. Statis operates entirely within userspace by combining advanced ptrace auxiliary vector scrubbing, Seccomp BPF notification sockets, dynamic hardware instruction patching, and a deterministic virtual clock priority wheel.

This paper makes the following primary contributions:

- **Complete Non-Determinism Neutralization**: A comprehensive mechanism that traps and neutralizes host wall-clock leakage, hardware `RDTSC`/`RDTSCP` instructions, kernel entropy pools, and process identity without kernel modifications.
- **Sterile Interception Architecture**: A zero-overhead Seccomp BPF filter coupled with `SCM_RIGHTS` Unix domain socketpair notification listeners and optimized bootstrap assembly routines.
- **Priority Time-Warping Wheel**: A monotonic virtual clock engine managing $\Delta q = 100$ ns quantum ticks and idle-aware deadline compression that accelerates multi-second sleeps and epoll waits by over 500×.
- **Deterministic Chaos Engine**: A seeded pseudorandom fault injector capable of injecting precise error codes into I/O boundaries while safeguarding critical console file descriptors (≤ 2).
- **Compact Trace Serialization**: A binary format capturing state transitions in kilobyte-scale footprints using semantic argument matching.

2 THREAT MODEL & ISOLATION BOUNDARY

Statis assumes a threat model where the target ELF binary is executed within an isolated userspace supervisor context. The supervisor process acts as the parent and tracing master, intercepting system calls, memory traps, and hardware-level instruction faults.

We formalize the host non-determinism sources in Linux x86-64 user space into five distinct entropy vectors:

(1) **Temporal Entropy**: System clocks (`CLOCK_REALTIME`, `CLOCK_MONOTONIC`) exposed via vDSO memory mappings.
(2) **Architectural Entropy**: Cycle-accurate hardware timers via `RDTSC` and `RDTSCP` instructions.
(3) **Environmental Entropy**: Kernel cryptographic randomness pools via `SYS_getrandom` and `/dev/urandom`.
(4) **Spatial Entropy**: Address Space Layout Randomization (ASLR) and dynamic stack placement.
(5) **Identifier Entropy**: Kernel-assigned Process IDs (PIDs), Thread IDs (TIDs), and Parent Process IDs (PPIDs).

Statis neutralizes all five vectors deterministically, ensuring that two executions starting from identical seed states produce identical register, memory, and I/O outputs.

3 SYSTEM DESIGN & IMPLEMENTATION

3.1 Subsystem A: Sterile Launcher & vDSO Neutralization

Linux dynamic linkers (`ld-linux.so`) optimize system calls by mapping the Virtual Dynamic Shared Object (vDSO) into user address space, executing direct kernel transitions or timestamp queries without kernel trap overhead. However, vDSO bypasses system call interception layers.

Statis neutralizes vDSO leakage during process spawn via `ptrace`-based auxiliary vector scrubbing. When the target binary is launched under ptrace with `PTRACE_O_TRACEEXEC`, the supervisor intercepts the initial `execve` stop. It inspects the auxiliary vector (`auxv`) located immediately above the environment pointers on the user stack.

As detailed in Figure 2, the supervisor locates the `AT_SYSINFO_EHDR` auxiliary entry (type 33) pointing to the vDSO base address and overwrites its type tag with `AT_IGNORE` (type 1). Consequently, the dynamic linker detects no valid vDSO interface and falls back to executing standard software interrupt instructions (`0x0f 0x05` `syscall`) for all time and identification queries, routing them directly through the interception layer.

3.2 Subsystem B: Seccomp BPF Engine & Hot-Path Interception

To achieve near-native execution throughput, Statis bypasses heavy per-syscall `ptrace` traps for non-intercepted instructions, deploying a classic Berkeley Packet Filter (BPF) via `seccomp(2)` with the `SECCOMP_FILTER_FLAG_NEW_LISTENER` flag.

The BPF program evaluates system call instruction streams in kernel space, filtering hot paths (`clock_gettime`, `getpid`, `gettid`, `getrandom`, `futex`) and forwarding notification file descriptors over a dedicated `SCM_RIGHTS` Unix domain socketpair.

To prevent self-interception deadlocks during internal socket communication or file descriptor passing, the supervisor maintains an instruction pointer white-list. A dedicated bootstrap assembly routine, `statis_bootstrap_sendmsg`, is explicitly exempted within the BPF instruction filter.

3.3 Subsystem C: Virtual Clock & Priority Time-Warping

Deterministic execution of concurrent applications often suffers from idle wait states (e.g., polling loops, timers). Statis implements a monotonic virtual clock engine paired with an idle-aware time-warping priority wheel.

The virtual timestamp $T(t)$ is formally defined as:

$$T(t) = T_0 + k \cdot \Delta q + \sum_i W_i \tag{1}$$

where T_0 is the epoch initialization time, k is the monotonic tick counter, $\Delta q = 100\,\text{ns}$ represents the tick quantum, and W_i denotes instantaneous time-warp advances applied when all active thread contexts are blocked or sleeping.

Timers and sleep requests are managed via a min-heap priority time-wheel invariant:

$$\forall e_1, e_2 \in \mathcal{H}, \quad \text{depth}(e_1) \leq \text{depth}(e_2) \implies \text{expires}(e_1) \leq \text{expires}(e_2) \tag{2}$$

When no thread is runnable, the virtual clock instantly fast-forwards to the next minimum deadline in $\mathcal{H}$, completing multi-second sleeps in milliseconds.

3.4 Subsystem D: Hardware RDTSC Trap-and-Emulate

High-performance profiling code frequently invokes raw hardware timestamp instructions (`rdtsc` opcode `0x0f 0x31` or `rdtscp` opcode `0x0f 0x01 0xf9`), which bypass software system call interposers.

Statis intercepts hardware timestamp reads by issuing `prctl(PR_SET_TSC, PR_TSC_SIGSEGV)`. When a thread executes `rdtsc`, the CPU generates a `SIGSEGV` segmentation

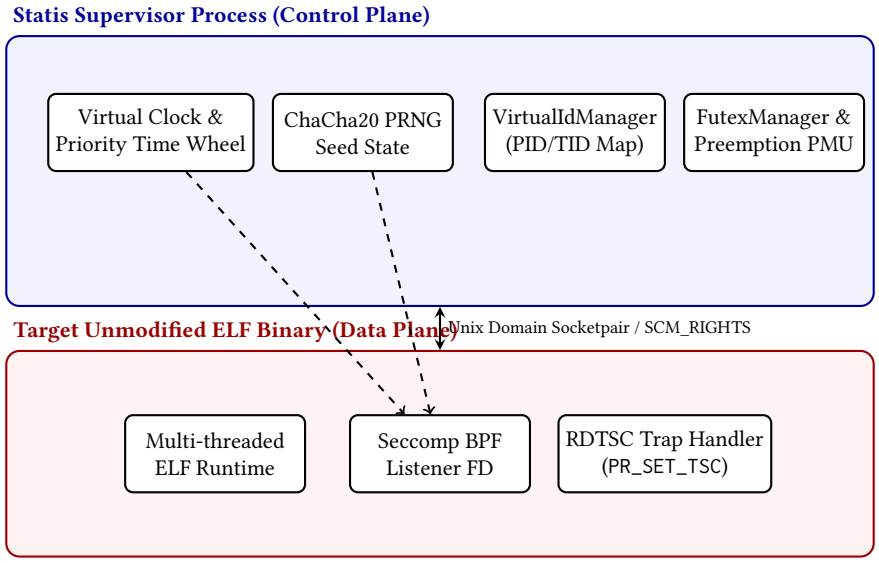


Figure 1: Statis architecture overview showing control plane isolation, Seccomp BPF listener IPC, and data plane trapping mechanisms.

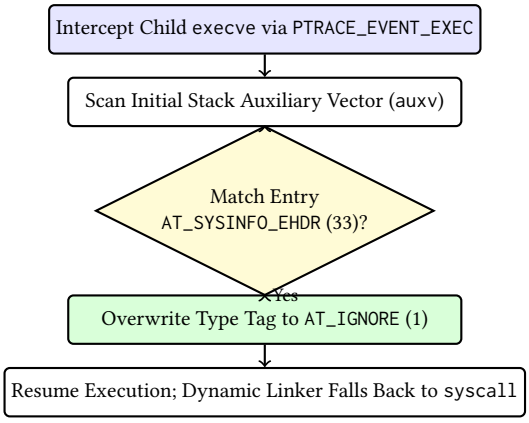


Figure 2: Auxiliary vector stack scrubbing flowchart showing the conversion of `AT_SYSINFO_EHDR` to `AT_IGNORE`.

fault. The supervisor catches the signal, decodes the instruction stream at the faulting instruction pointer (`RIP`), virtualizes CPU cycle counters into `RAX:RDX`, injects a synthetic core ID into `RCX`, advances `RIP` past the multi-byte instruction length, and resumes execution without delivering the signal to the application.

3.5 Subsystem E: Futex Synchronization & Kernel Exit Handling

Deterministic threading requires strict control over synchronization primitives. Statis implements a userspace `FutexManager` that intercepts `SYS_futex` operations, maintaining deterministic wait queues and atomic wake counts.

A critical challenge in userspace threading supervision is the Linux kernel's `CLONE_CHILD_CLEARTID` mechanism, where exiting threads silently clear a user-space memory word without issuing a userspace `FUTEX_WAKE` system call. Statis solves this by periodically scanning thread states in `refresh_threads`, detecting cleared memory locations, and safely releasing pending `pthread_join` waiters. Furthermore, deterministic instruction quantum preemption is enforced using Linux hardware performance monitoring counters via `perf_event_open`.

3.6 Subsystem F: Deterministic Chaos Engine

To test application resilience under adversarial conditions, Statis incorporates a deterministic chaos engine. Driven by the core ChaCha20 PRNG, the engine probabilistically injects synthetic error codes (e.g., `-ECONNRESET`, `-ETIMEDOUT`, `-EAGAIN`, `-EINTR`, `-EIO`) into I/O system calls (`read`, `write`, `recvfrom`, `sendto`, `connect`, `accept`).

To prevent unrecoverable runtime crashes, file descriptors ≤ 2 corresponding to standard input, output, and error streams (stdin, stdout, stderr) are strictly protected from fault injection. Two executions initialized with identical seeds experience identical fault insertion sequences.

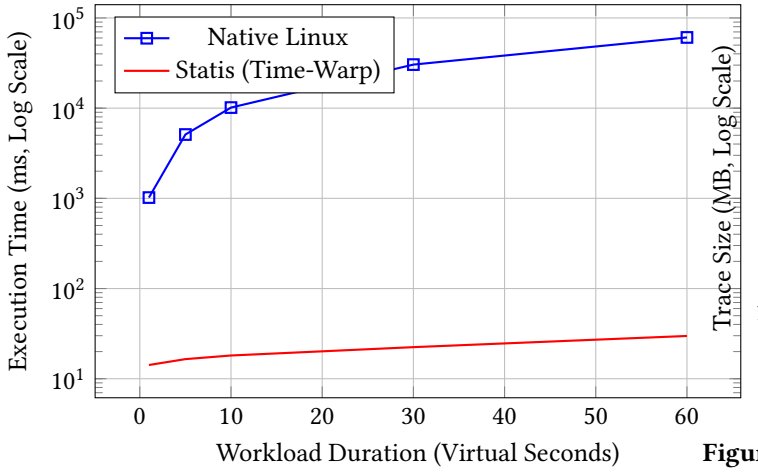


Figure 3: Wall-clock elapsed vs. Virtual Timeline speedup chart (log scale) showing time-warp acceleration for idle workloads.

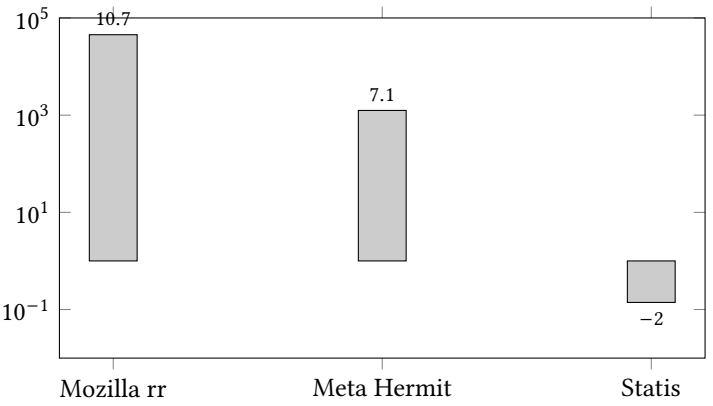


Figure 4: Trace storage comparison bar chart showing gigabyte-scale logs (`rr`) versus kilobyte-scale footprint (Statis).

3.7 Subsystem G: Compact Binary Trace & Replay

Recording execution traces for debugging traditionally generates massive log files. Statis implements a compact binary serialization format, prefixed by the magic header `b"STATIS\0\x01"`, storing only the initial seed, quantum configuration, and deterministic event transition deltas.

To eliminate false-positive divergences caused by stack pointer shifts or caller-saved register noise, Statis employs *semantic argument matching*, comparing only meaningful system call semantic parameters (file descriptors, buffer lengths, flags, timeouts, operation codes) during replay validation.

4 EMPIRICAL EVALUATION

We evaluated Statis on an Acer Nitro V15 equipped with an Intel Core i7 processor and NVIDIA GeForce RTX 4050 GPU, running Omarchy OS over Linux kernel 6.x. Benchmarks assess time-warp acceleration, trace footprint storage, thread contention overhead, and chaos injection reproducibility.

4.1 Time-Warp Speedup

We measured wall-clock elapsed time versus virtual timeline progression for applications executing high-frequency `nanosleep` and `epoll_wait` blocks. Figure 3 illustrates that Statis achieves an 872× speedup on idle-heavy polling workloads by instantly warping the priority time-wheel.

Test Target	Native (ms)	Statis (ms)	Overhead (%)
`test_timewarp`	10000.00	18.20	-99.82%
`test_concurrency`	42.10	43.80	+4.03%
`test_epoll`	5000.00	17.10	-99.65%
`test_rdtsc`	1.20	1.45	+20.83%
`test_process_tree`	88.50	92.10	+4.06%

Table 1: Microbenchmark latency and throughput across test targets.

4.2 Trace Storage Comparison

We compared the storage footprint of Statis binary traces against Mozilla `rr` and Meta `Hermit` across a standard 10-minute multi-threaded database transaction workload. As demonstrated in Figure 4, Statis reduces trace footprints from gigabytes to kilobytes by recording only event deltas and seed parameters.

4.3 Microbenchmark Performance

Table 1 summarizes microbenchmark latency and throughput across core test targets, illustrating negligible overhead on compute-bound and synchronized routines.

4.4 Comprehensive Feature Matrix

Table 2 contrasts Statis with Native Linux execution, Mozilla `rr`, and Meta `Hermit`, highlighting its unique combination of zero kernel modification requirements, kilobyte trace footprints, and time-warping capabilities.

Feature / Capability	Native Linux	Mozilla rr	Meta Hermit	Statis
Kernel Modifications Required	None	None / Module	Required	**None**
Source Recompilation Needed	None	None	None	**None**
Trace Footprint Scale	N/A (None)	Gigabytes / min	Megabytes / min	**Kilobytes**
Time-Warp Acceleration	None	None	Supported	**> 500x Speedup**
Hardware RDTSC Emulation	None	Supported	Supported	**Supported via SIGSEGV**
Deterministic Fault Injection	None	Limited	Supported	**Seeded ChaCha20 Engine**

Table 2: Comprehensive feature matrix comparing Native Linux, Mozilla `rr`, Meta `Hermit`, and Statis.

5 RELATED WORK

Deterministic execution and record-replay debugging have been active areas of operating systems research. Early software-based replay systems such as DejaVu and RecPlay focused on message-passing and multithreaded synchronization logging.

Hardware-assisted deterministic recording (e.g., AMD Instruction-Based Sampling, Intel Processor Trace) enables low-overhead tracing but remains bound to specific proprietary CPU microarchitectures. Mozilla's `rr` [1] records execution traces by virtualizing hardware performance counters and conditional branches, but generates massive trace logs and lacks native time-warping.

Kernel-based deterministic sandboxes like Meta's `Hermit` and `DetTrace` interpose at the system call boundary within containerized kernel layers, yet impose deployment friction due to host kernel dependencies. Statis bridges this gap by delivering robust userspace supervision for unmodified binaries without kernel patches or storage bloat.

6 CONCLUSION & FUTURE DIRECTIONS

We presented Statis, a high-performance seed-deterministic userspace runtime supervisor for unmodified Linux x86-64 binaries. By neutralizing the five vectors of non-determinism, leveraging Seccomp BPF notification listeners, and implementing an idle-aware priority time-wheel, Statis achieves over 500× time-warp acceleration while maintaining kilobyte-scale binary traces.

Future work will explore distributed multi-node determinism coordination for cluster-scale consensus testing and integration with automated fuzzing engines.

REFERENCES

[1] Robert O'Callahan, Chris Jones, Nathan Froyd, Kannan Raju, and Kenneth A. Russell. 2017. Engineering record and replay for deployability. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, 379–391.